```
#
 ↪  ================================================================================
# Advanced Equation of State Comparison and Residual Properties
# Description:
#   This interactive notebook calculates and compares the
 ↪  isotherms, volume roots,
#   and residual properties (U^R, H^R, S^R) for the van der
 ↪  Waals, SRK, and
#   Peng-Robinson equations of state against real fluid data
 ↪  from CoolProp.
#
 ↪  ================================================================================

# Quietly install CoolProp if running on Google Colab
try:
    import CoolProp
except ImportError:
    import subprocess
    import sys
    subprocess.check_call([sys.executable, "-m", "pip",
 ↪  "install", "-q", "CoolProp"])
    print("CoolProp successfully installed.")

import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output
import CoolProp.CoolProp as CP

# Dictionary mapping friendly names to CoolProp string
 ↪  identifiers
fluid_dict = {
    'Methane': 'Methane',
    'Ethane': 'Ethane',
    'Propane': 'Propane',
    'n-Hexane': 'n-Hexane',
    'n-Octane': 'n-Octane',
    'Ethanol': 'Ethanol',
    'Nitrogen': 'Nitrogen',
    'Oxygen': 'Oxygen',
    'Ammonia': 'Ammonia',
    'Carbon Dioxide': 'CarbonDioxide',
    'Carbon Monoxide': 'CarbonMonoxide',
    'Argon': 'Argon',
    'Water': 'Water',
    'R-32': 'R32'
}
```

```
def compute_all_eos_and_plot(fluid_name, T_val, T_unit, P_val,
 ↪  P_unit):
    fluid = fluid_dict[fluid_name]

    # --- 1. Unit Conversions ---
    if T_unit == '°C': T = T_val + 273.15
    elif T_unit == '°F': T = (T_val - 32) * 5/9 + 273.15
    else: T = T_val

    if P_unit == 'atm': P = P_val * 1.01325
    elif P_unit == 'Pa': P = P_val / 1e5
    elif P_unit == 'MPa': P = P_val * 10
    elif P_unit == 'psi': P = P_val / 14.5038
    else: P = P_val

    if T <= 0 or P <= 0:
        print("Temperature and Pressure must be greater than
         ↪  absolute zero.")
        return

    # --- 2. Fetch Critical Properties & Real Volume ---
    Tc = CP.PropsSI('TCRIT', fluid)
    Pc = CP.PropsSI('PCRIT', fluid) / 1e5  # Convert Pa to bar
    omega = CP.PropsSI('ACENTRIC', fluid)

    R_Lbar = 0.0831446 # L*bar/(mol*K)
    R_J = 8.31446      # J/(mol*K)
    V_ideal = R_Lbar * T / P

    # Attempt to get Vapor Pressure from CoolProp (if
     ↪  subcritical)
    P_sat_bar = None
    if T < Tc:
        try:
            P_sat_bar = CP.PropsSI('P', 'T', T, 'Q', 0, fluid) /
 ↪  1e5
        except ValueError:
            pass

    # Attempt to get real volume roots from CoolProp
    v_real = []
    try:
        # Check if the fluid is in the two-phase region
        phase = CP.PhaseSI('T', T, 'P', P*1e5, fluid)
        if 'twophase' in phase.lower():
            rho_l = CP.PropsSI('Dmolar', 'T', T, 'Q', 0, fluid)
            rho_v = CP.PropsSI('Dmolar', 'T', T, 'Q', 1, fluid)
            v_real = [1/rho_l * 1000, 1/rho_v * 1000] # Convert
 ↪  to L/mol
```

```
        else:
            rho = CP.PropsSI('Dmolar', 'T', T, 'P', P*1e5,
↪   fluid)
            v_real = [1/rho * 1000]
    except ValueError:
        pass # CoolProp failed to converge or out of bounds

    # --- 3. Setup Models ---
    eos_data = {}
    Tr = T / Tc

    # Model 1: van der Waals
    a_vdw = 27 * (R_Lbar * Tc)**2 / (64 * Pc)
    b_vdw = R_Lbar * Tc / (8 * Pc)
    C2_vdw = -(b_vdw + R_Lbar * T / P)
    C1_vdw = a_vdw / P
    C0_vdw = -a_vdw * b_vdw / P
    r_vdw = np.roots([1, C2_vdw, C1_vdw, C0_vdw])
    roots_vdw = np.sort(r_vdw[np.isclose(r_vdw.imag, 0)].real)
    roots_vdw = roots_vdw[roots_vdw > b_vdw]
    eos_data['vdW'] = {'roots': roots_vdw, 'a': a_vdw, 'b':
↪   b_vdw, 'da_dT': 0, 'color': 'green'}

    # Model 2: SRK
    ac_srk = 0.42748 * (R_Lbar**2 * Tc**2) / Pc
    b_srk = 0.08664 * R_Lbar * Tc / Pc
    m_srk = 0.480 + 1.574 * omega - 0.176 * omega**2
    alpha_srk = (1 + m_srk * (1 - np.sqrt(Tr)))**2
    a_srk = ac_srk * alpha_srk
    da_dT_srk = -ac_srk * m_srk * np.sqrt(alpha_srk / (T * Tc))
    C2_srk = -(R_Lbar * T / P)
    C1_srk = (a_srk / P) - b_srk**2 - (R_Lbar * T * b_srk / P)
    C0_srk = -(a_srk * b_srk / P)
    r_srk = np.roots([1.0, C2_srk, C1_srk, C0_srk])
    roots_srk = np.sort(r_srk[np.abs(r_srk.imag) < 1e-9].real)
    roots_srk = roots_srk[roots_srk > b_srk]
    eos_data['SRK'] = {'roots': roots_srk, 'a': a_srk, 'b':
↪   b_srk, 'da_dT': da_dT_srk, 'color': 'blue'}

    # Model 3: Peng-Robinson
    ac_pr = 0.45724 * (R_Lbar**2 * Tc**2) / Pc
    b_pr = 0.07780 * R_Lbar * Tc / Pc
    m_pr = 0.37464 + 1.54226 * omega - 0.26992 * omega**2
    alpha_pr = (1 + m_pr * (1 - np.sqrt(Tr)))**2
    a_pr = ac_pr * alpha_pr
    da_dT_pr = -ac_pr * m_pr * np.sqrt(alpha_pr / (T * Tc))
    C2_pr = b_pr - (R_Lbar * T / P)
    C1_pr = (a_pr / P) - 2 * b_pr * (R_Lbar * T / P) - 3 *
↪   b_pr**2
```

```
    C0_pr = b_pr**3 + (R_Lbar * T / P) * b_pr**2 - (a_pr * b_pr
↪   / P)
    r_pr = np.roots([1.0, C2_pr, C1_pr, C0_pr])
    roots_pr = np.sort(r_pr[np.abs(r_pr.imag) < 1e-9].real)
    roots_pr = roots_pr[roots_pr > b_pr]
    eos_data['PR'] = {'roots': roots_pr, 'a': a_pr, 'b': b_pr,
↪   'da_dT': da_dT_pr, 'color': 'red'}

    # --- 4. Print Outputs ---
    print("="*85)
    print(f"SYSTEM: {fluid_name}  |  Critical State: Tc =
     ↪  {Tc:.2f} K, Pc = {Pc:.2f} bar, ω = {omega:.4f}")

    if P_sat_bar is not None:
        print(f"TARGET: T = {T:.2f} K  |  P = {P:.2f} bar  |
         ↪  V_ideal = {V_ideal:.4f} L/mol  |  Psat =
         ↪  {P_sat_bar:.2f} bar")
    else:
        print(f"TARGET: T = {T:.2f} K  |  P = {P:.2f} bar  |
         ↪  V_ideal = {V_ideal:.4f} L/mol  |  Psat = N/A
         ↪  (Supercritical)")

    if v_real:
        print(f"TRUE VOLUMES (CoolProp): {[round(v, 4) for v in
         ↪  v_real]} L/mol")
    print("="*85)

    for name, data in eos_data.items():
        print(f"\n[{name} EQUATION OF STATE]")
        for i, root in enumerate(data['roots']):
            # Calculate Residuals
            a, b, da_dT = data['a'], data['b'], data['da_dT']
            V_R = root - V_ideal

            # Unstable root skip
            if len(data['roots']) == 3 and i == 1:
                print(f"  Root {i+1}: V = {root:.4f} L/mol
                 ↪  (Unstable Phase)")
                continue

            if name == 'vdW':
                U_R = (-a / root) * 100
                S_R = R_J * np.log(P * (root - b) / (R_Lbar *
↪   T))
            elif name == 'SRK':
                log_term = np.log((root + b) / root)
                U_R = 100 * (T * da_dT - a) / b * log_term
```

```
                S_R = R_J * np.log(P * (root - b) / (R_Lbar *
↪  T)) + 100 * (da_dT / b) * log_term
            elif name == 'PR':
                log_term = np.log((root + (1 + np.sqrt(2))*b) /
↪  (root + (1 - np.sqrt(2))*b))
                U_R = 100 * (T * da_dT - a) / (2 * np.sqrt(2) *
↪  b) * log_term
                S_R = R_J * np.log(P * (root - b) / (R_Lbar *
↪  T)) + 100 * (da_dT / (2 * np.sqrt(2) * b)) * log_term

            H_R = U_R + 100 * P * V_R

            phase_label = "Stable Phase" if len(data['roots'])
↪  == 1 else ("Liquid Root" if i == 0 else "Vapor Root")
            print(f"  Root {i+1}: V = {root:.4f} L/mol
             ↪  ({phase_label})")
            print(f"      U^R = {U_R:>8.1f} J/mol  |  H^R =
             ↪  {H_R:>8.1f} J/mol  |  S^R = {S_R:>7.3f}
             ↪  J/(mol·K)")

    print("\n" + "="*85)

    # --- 5. Plotting ---
    b_max = max(b_vdw, b_srk, b_pr)
    V_max = V_ideal * 1.5
    for data in eos_data.values():
        if len(data['roots']) == 3:
            V_max = max(V_max, data['roots'][2] * 1.3)

    V_arr = np.linspace(b_max * 1.05, V_max, 2000)

    # Isotherms
    P_iso_vdw = (R_Lbar * T) / (V_arr - b_vdw) - a_vdw /
↪  (V_arr**2)
    P_iso_srk = (R_Lbar * T) / (V_arr - b_srk) - a_srk / (V_arr
↪  * (V_arr + b_srk))
    P_iso_pr  = (R_Lbar * T) / (V_arr - b_pr) - a_pr / (V_arr**2
↪  + 2*b_pr*V_arr - b_pr**2)

    plt.figure(figsize=(11, 7))
    plt.plot(V_arr, P_iso_vdw, color=eos_data['vdW']['color'],
↪  lw=2, label='van der Waals Isotherm')
    plt.plot(V_arr, P_iso_srk, color=eos_data['SRK']['color'],
↪  lw=2, label='SRK Isotherm')
    plt.plot(V_arr, P_iso_pr, color=eos_data['PR']['color'],
↪  lw=2, label='Peng-Robinson Isotherm')
```

```
    plt.axhline(P, color='k', linestyle=':', lw=1.5,
 ↪  label=f'Target Pressure ({P:.1f} bar)')

    # Plot roots
    for name, data in eos_data.items():
        for i, root in enumerate(data['roots']):
            marker = 'x' if (len(data['roots']) == 3 and i == 1)
 ↪  else 'o'
            plt.plot(root, P, marker=marker,
 ↪  color=data['color'], markersize=8,
                     label=f'{name} Root' if i==0 else "")

    # Plot True CoolProp Roots
    for i, v in enumerate(v_real):
        plt.plot(v, P, marker='*', color='k', markersize=12,
 ↪  label='True Vol (CoolProp)' if i==0 else "")

    plt.ylim(0, max(P * 1.5, Pc * 1.5))
    plt.xlim(0, V_max)
    plt.xlabel('Molar Volume, V (L/mol)', fontsize=12)
    plt.ylabel('Pressure, P (bar)', fontsize=12)
    plt.title(f'Equation of State Comparison for {fluid_name} at
 ↪  {T:.1f} K', fontsize=14, fontweight='bold')

    # Prevent duplicate legend labels
    handles, labels = plt.gca().get_legend_handles_labels()
    by_label = dict(zip(labels, handles))
    plt.legend(by_label.values(), by_label.keys(), loc='upper
 ↪  right')

    plt.grid(True, linestyle='--', alpha=0.6)
    plt.show()

# --- 6. Interactive UI Setup ---
style = {'description_width': 'initial'}

fluid_dropdown =
 ↪  widgets.Dropdown(options=list(fluid_dict.keys()),
 ↪  value='Methane', description='Fluid:', style=style)

T_input = widgets.FloatText(value=300,
 ↪  description='Temperature:', style=style,
 ↪  layout=widgets.Layout(width='200px'))
T_unit = widgets.Dropdown(options=['K', '°C', '°F'], value='K',
 ↪  layout=widgets.Layout(width='80px'))

P_input = widgets.FloatText(value=100, description='Pressure:',
 ↪  style=style, layout=widgets.Layout(width='200px'))
```

```
P_unit = widgets.Dropdown(options=['bar', 'atm', 'Pa', 'MPa',
 ↪  'psi'], value='bar', layout=widgets.Layout(width='80px'))

T_box = widgets.HBox([T_input, T_unit])
P_box = widgets.HBox([P_input, P_unit])

ui = widgets.VBox([fluid_dropdown, T_box, P_box])
out = widgets.interactive_output(compute_all_eos_and_plot, {
    'fluid_name': fluid_dropdown,
    'T_val': T_input,
    'T_unit': T_unit,
    'P_val': P_input,
    'P_unit': P_unit
})

display(ui, out)
```

```
VBox(children=(Dropdown(description='Fluid:', options=('Methane', 'Ethane',
Hexane', 'n-Octane',…

Output()
```